Hacettepe University — Computer Engineering
CMP 720 EMBEDDED SYSTEMS DESIGN

Spring 2026 — Mini In-Class Project Series Part 3 - SAMPLE SOLUTION

Lab 3: Sensor Interfacing, Drivers & Hard Timing

SafePulse — Embedded Patient Monitoring System

Week 4: Sensor & Actuator Interfacing + Timing + Drivers

Duration:	1 – 1.5 hours
Tools Required:	Renode, ARM GCC toolchain, terminal, Week 2 spec
Deliverable:	Firmware that reads real sensors and meets the 100 ms alarm deadline
Prerequisites:	Completed Lab 2 (Week 3) — working UART & GPIO

You will need to look things up during this lab. All documents are freely available online.

STM32F4 Reference Manual (RM0090) — *the primary reference for everything hardware-related.*

Renode documentation — `https://renode.readthedocs.io`

Using Python in Renode — `https://renode.readthedocs.io/en/latest/basic/using-python.html`
The `Mocks.DummyI2CSlave` peripheral provides a set of methods and events that allow you to implement a simple I2C peripheral. See `Dummy I2C Slave`

Sensor datasheets — MAX30102 datasheet (HR/SpO_2); TMP102 datasheet (temperature). Device I2C addresses are defined in these documents.

1 Overview

This is the **critical week** of the SafePulse project. You will replace the hardcoded sensor values from Week 3 with a real I2C sensor, and replace the busy-wait delay loop with a **hardware timer interrupt** that fires every 200 ms. Most importantly, you will implement the safety-critical alarm logic and verify that your system meets the **100 ms hard real-time deadline** from your Week 2 specification.

Recall NFR-01: Alarm activation latency from threshold breach detection to GPIO assertion shall not exceed **100 ms** under any system load. Today, you must demonstrate this is achieved.

1.1 Learning Objectives

1. Write a sensor driver that reads registers from an I2C peripheral using the provided I2C driver functions.

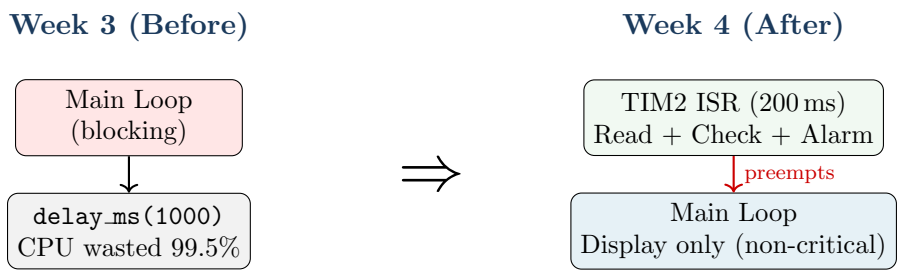
2. Configure a hardware timer (TIM2) for periodic interrupt generation at a precise 200 ms interval.

3. Implement an Interrupt Service Routine (ISR) for real-time sensor sampling.

4. Implement threshold-based alarm logic and verify hard real-time deadline compliance.

5. Understand the ISR vs. main-loop architecture and why it enables real-time guarantees.

1.2 Architecture: Before and After



2 The Vital Signs Sensor (I2C)

A vital signs sensor is connected to the STM32F4 via the **I2C1 bus** (PB6=SCL, PB7=SDA) at 7-bit address `0x55`. We chose this address to avoid conflicts with common I2C devices on the STM32F4-Discovery board (real temperature sensors like the TMP102 default to `0x48`; EEPROMs use `0x50--0x57`). In a real product, the sensor address would be defined in the manufacturer's datasheet. In Renode, no physical sensor exists, so we need a software model that responds to I2C transactions. Renode provides `DummyI2CSlave` — a generic I2C slave peripheral that accepts reads and writes on the bus. We attach Python hooks (`vitals_sensor.py`) to this peripheral to implement our register logic and the deterioration scenario. The alternative would be writing a full C# peripheral class compiled into Renode; `DummyI2CSlave` with Python hooks achieves the same result with far less setup, making it practical for lab use.

Hardware portability: The I2C driver code (`i2c_init`, `i2c_read_register`, `i2c_write_register`) uses standard STM32F4 register addresses from the RM0090 reference manual and works identically in Renode and on real hardware. The sensor-specific code (`read_vitals`, register addresses, thresholds) would need to be adapted to match a real sensor's datasheet — this is normal and expected in embedded development.

2.1 I2C Sensor Register Map

Like any I2C sensor, our device exposes its data through numbered **internal registers**. Our simulated sensor uses a **simplified register interface** designed for this lab. Real I2C sensors have more complex data formats — for example, the TMP102 temperature sensor stores temperature as a 12-bit left-justified value in a 16-bit register with 0.0625 °C resolution, requiring bit-shifting and

scaling in firmware. Similarly, a real pulse oximeter like the MAX30102 outputs raw photodiode data, not a direct SpO2 percentage — computing SpO2 requires DSP algorithms beyond the scope of this lab.

Our sensor instead provides pre-processed, human-readable values (BPM, percentage, degrees) so you can focus on the I2C bus protocol, timer interrupts, and alarm logic. The register layout follows common conventions:

- **Data registers (0x00–0x03):** Data registers are grouped at low addresses for efficient access. The two temperature registers (integer + fractional tenths) are a simplification of the multi-byte fixed-point format used by sensors like the TMP102 and LM75.
- **Device ID (0x04):** Returns a fixed value (`0xA5`). Most I2C sensors include an identification register for firmware to verify the correct device is present on the bus (e.g., the MAX30102 returns `0x15` in its Part ID register at address `0xFF`).
- **Scenario control (0x05):** This register is specific to our simulation — writing `0x01` starts a pre-programmed patient deterioration sequence. A real sensor would not have this register; vital signs would come from an actual patient.

To read a sensor register, the firmware performs a standard I2C combined transaction: write the register address, then read one byte. The provided `i2c_read_register()` and `i2c_write_register()` functions in the starter code handle this protocol for you.

Register	Name	Access	Description
`0x00`	Heart Rate	Read	Heart rate in BPM (uint8)
`0x01`	SpO2	Read	Blood oxygen saturation in % (uint8)
`0x02`	Temp (int)	Read	Temperature integer part in °C (uint8)
`0x03`	Temp (frac)	Read	Temperature fractional part in tenths (uint8)
`0x04`	Device ID	Read	Always returns `0xA5` (sensor presence check)
`0x05`	Scenario	Write	Write `0x01` to start deterioration (simulation only)

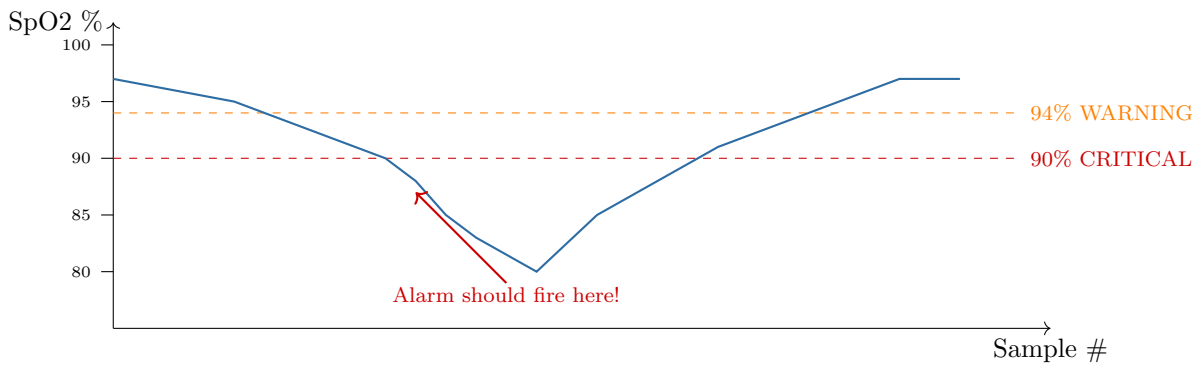
Example usage with the provided driver functions:

```
/* Read one register from the sensor */
uint8_t spo2 = i2c_read_register(SENSOR_I2C_ADDR, SENSOR_REG_SPO2);

/* Write to the scenario control register */
i2c_write_register(SENSOR_I2C_ADDR, SENSOR_REG_SCENARIO, 0x01);
```

2.2 Built-in Deterioration Scenario

When you write `0x01` to register `0x05`, the sensor enters a pre-programmed scenario where the patient's SpO2 gradually drops from 97% down to 80% and then recovers. The timeline is:



Your system should detect when SpO2 crosses below 90% and fire the alarm **within 100 ms of detection**. With a 200 ms polling period, detection itself can occur up to one sample interval after the physiological event. However, once a problematic value is read, the alarm is asserted in the same ISR execution (typically in microseconds), which satisfies the 100 ms detection-to-alarm requirement.

3 Provided Code

The `main.c` skeleton provides several complete subsystems that you should study but **not modify**:

3.1 I2C Driver (Provided Complete)

The I2C driver configures I2C1 in standard mode (100 kHz) on PB6 (SCL) and PB7 (SDA). Three functions are provided:

- `i2c_init()` — Enables clocks, configures GPIO pins as alternate function open-drain with pull-ups, and sets up the I2C peripheral (prescaler, clock control, rise time).
- `i2c_read_register(dev_addr, reg_addr)` — Performs a combined I2C write-then-read transaction: sends the register address, then reads one byte. Returns the byte value.
- `i2c_write_register(dev_addr, reg_addr, value)` — Writes one byte to a register address.

How the I2C read works internally:
`START` → `addr(Write)` → `register` → `RESTART` → `addr(Read)` → `data` → `STOP`

The firmware first sends the sensor address with the write bit to select which register to read, then issues a repeated start and sends the address again with the read bit to receive the data byte. The provided driver polls four status flags in `I2C1_SR1` after each bus event:

- **SB** (Start Bit) — set when a START or RESTART condition has been generated
- **ADDR** (Address Sent) — set when the slave has acknowledged its address
- **BTF** (Byte Transfer Finished) — set when a data byte has been fully transmitted
- **RXNE** (Receive Not Empty) — set when a received byte is ready to read from `I2C1_DR`

See RM0090 Section 27.3.3 for the full state machine diagram.

3.2 GPIO and Alarm Functions

The GPIO driver configures PD12 (green LED as an extra output) and PD14 (red LED / alarm) as outputs (note the change in alarm LED from Week 3). The `GPIOD_MODER` register uses 2 bits per pin to select the mode (`00`=input, `01`=output, `10`=alternate function, `11`=analog). The `GPIOD_ODR` register uses 1 bit per pin to control the output level.

Two helper functions are provided for the alarm:

- `alarm_on()` — Sets PD14 high (red LED on)
- `alarm_off()` — Sets PD14 low (red LED off)

4 Implementation Tasks

The provided `main.c` skeleton contains four TODO sections. Complete them in order.

4.1 Task 1: Read Vital Signs from Sensor (`read_vitals`)

Implement the function that reads all sensor registers via I2C and stores them in global variables. Verify the device ID first to detect sensor faults.

Listing 1: TODO 1: Sensor read function

```c
uint8_t read_vitals(void) {
    // IMPORTANT: Always verify the sensor is present BEFORE
    // reading data.  If the sensor is disconnected or faulty,
    // i2c_read_register() will return garbage -- not 0xA5.
    //
    // a) Read SENSOR_REG_ID via I2C, verify == 0xA5
    //    If not, return 0 (sensor fault)
    // b) Read SENSOR_REG_HR -> latest_hr
    // c) Read SENSOR_REG_SPO2 -> latest_spo2
    // d) Read SENSOR_REG_TEMP_INT -> latest_temp_int
    // e) Read SENSOR_REG_TEMP_FRC -> latest_temp_frac
    // f) Return 1 (success)
    //
    // Each read is one call:
    //   val = i2c_read_register(SENSOR_I2C_ADDR, register);
}
```

```c
uint8_t read_vitals(void) {
    /* Verify sensor presence via device ID */
    if (i2c_read_register(SENSOR_I2C_ADDR, SENSOR_REG_ID) != 0xA5) {
        return 0;  /* Sensor fault */
    }

    /* Read all sensor registers via I2C */
    latest_hr        = i2c_read_register(SENSOR_I2C_ADDR,
        SENSOR_REG_HR);
    latest_spo2      = i2c_read_register(SENSOR_I2C_ADDR,
        SENSOR_REG_SPO2);
    latest_temp_int  = i2c_read_register(SENSOR_I2C_ADDR,
```

```
        SENSOR_REG_TEMP_INT);
    latest_temp_frac = i2c_read_register(SENSOR_I2C_ADDR,
        SENSOR_REG_TEMP_FRC);

    return 1;  /* Success */
}
```

Key points: The ID check happens first as a guard. If the sensor disconnects or malfunctions, the I2C read will return an unexpected value, and we return 0 to signal the fault. The global variables are declared `volatile` because they're shared between the ISR and main loop. Each `i2c_read_register()` call performs a full I2C bus transaction (start, address, register select, restart, read, stop).

4.2 Task 2: Threshold Checking (`check_thresholds`)

Implement the state machine logic from your Week 2 design. The critical requirement: when entering CRITICAL state, call `alarm_on()` **immediately** within the same function, do not defer it to lower-priority code.

The function should use these global variables and functions:

- **Inputs:** `latest_hr`, `latest_spo2`, `latest_temp_int`, `latest_temp_frac`
- **State:** `current_state` (type `system_state_t` — update the value of this variable)
- **Actions:** `alarm_on()`, `alarm_off()`

Temperature comparison tip: Combine integer and fractional parts into tenths for easy comparison:
`uint16_t temp_tenths = (uint16_t)latest_temp_int * 10 + latest_temp_frac;`
Example: 38.5 °C becomes `temp_tenths = 385`.

Check conditions in priority order. Each condition considers **all three** vital signs:

1. **CRITICAL** (check first): SpO2 < 90 **or** temp > 40.0 °C **or** HR < 40 **or** HR > 150
   → set `STATE_CRITICAL`, call `alarm_on()`
2. **WARNING**: SpO2 ≤ 94 **or** temp ≥ 38.0 °C **or** HR < 50 **or** HR > 120
   → set `STATE_WARNING`; if previously CRITICAL, call `alarm_off()`
3. **NORMAL**: all values in safe range
   → set `STATE_NORMAL`; if previously CRITICAL, call `alarm_off()`

Listing 2: TODO 2: Threshold checking function

```
void check_thresholds(void) {
    // Tip: combine temperature into tenths first:
    //   uint16_t temp_tenths = (uint16_t)latest_temp_int * 10
    //                          + latest_temp_frac;
    //
    // Check in priority order (CRITICAL first):
    //
    // a) If SpO2 < 90 OR temp > 40.0 OR HR < 40 OR HR > 150:
```

```
    //     -> Set current_state = STATE_CRITICAL
    //     -> Call alarm_on() IMMEDIATELY
    //     -> return
    //
    // b) If SpO2 <= 94 OR temp >= 38.0 OR HR < 50 OR HR > 120:
    //     -> If previous state was CRITICAL, call alarm_off()
    //     -> Set current_state = STATE_WARNING
    //     -> return
    //
    // c) Otherwise (all values safe):
    //     -> If previous state was CRITICAL, call alarm_off()
    //     -> Set current_state = STATE_NORMAL
}
```

```c
void check_thresholds(void) {
    uint16_t temp_tenths = (uint16_t)latest_temp_int * 10
                           + latest_temp_frac;

    /* CRITICAL check first (safety priority) */
    if (latest_spo2 < SPO2_CRITICAL
        || temp_tenths > TEMP_CRITICAL
        || latest_hr < HR_CRITICAL_LOW
        || latest_hr > HR_CRITICAL_HIGH) {
        if (current_state != STATE_CRITICAL) {
            current_state = STATE_CRITICAL;
            alarm_on();  /* IMMEDIATE - safety-critical path */
        }
        return;
    }

    /* WARNING check */
    if (latest_spo2 <= SPO2_WARNING
        || temp_tenths >= TEMP_WARNING
        || latest_hr < HR_WARNING_LOW
        || latest_hr > HR_WARNING_HIGH) {
        if (current_state == STATE_CRITICAL) {
            alarm_off();  /* Downgrade from critical */
        }
        current_state = STATE_WARNING;
        return;
    }

    /* All normal */
    if (current_state == STATE_CRITICAL) {
        alarm_off();
    }
    current_state = STATE_NORMAL;
}
```

Design note: This simplified version transitions immediately based on single readings. A pro-

duction system would use hysteresis (as designed in Week 2) — requiring multiple consecutive normal readings before downgrading from CRITICAL. The simplified version is used here to keep the lab within 1 hour; the RTOS version in Week 5 will add hysteresis.

4.3 Task 3: Timer Configuration (`timer_init`)

Configure TIM2 to generate an update interrupt every 200 ms. This is the core of the real-time guarantee.

How STM32 timers work:
The timer has a counter that increments at each "tick". The tick rate is set by the prescaler:

$$\text{tick_rate} = \frac{\text{timer_clock}}{\text{PSC} + 1}$$

The counter counts from 0 up to the auto-reload value (ARR). When it reaches ARR, it resets to 0 and fires an interrupt:

$$\text{interrupt_period} = \frac{\text{ARR} + 1}{\text{tick_rate}}$$

Both PSC and ARR use "value − 1" because the hardware counts inclusively from 0. For example, PSC = 2 means the hardware divides by 3 (counts 0, 1, 2 then ticks).

Timer math for 200 ms:
Step 1: Prescaler = 8400 − 1 = 8399 → tick_rate = 84,000,000 ÷ 8400 = 10,000 Hz (one tick = 0.1 ms)
Step 2: Auto-reload = 2000 − 1 = 1999 → period = 2000 ÷ 10,000 = 0.2 s = 200 ms ✓
Verify: 84,000,000 ÷ 8400 ÷ 2000 = 5 Hz = 200 ms period ✓

Two levels of interrupt enable:
You must enable the interrupt at **both** levels for it to fire:

1. **Peripheral level** (`TIM2_DIER`, UIE bit): Tells TIM2 to assert its interrupt line when the counter overflows. Without this, the timer counts but never signals anyone.
2. **CPU level** (`NVIC_ISER0`, bit 28): Tells the NVIC to deliver TIM2's interrupt to the CPU. Without this, TIM2 asserts the signal but the CPU ignores it.

If you forget either one, the ISR never fires. Forgetting the NVIC enable is a common mistake.

Listing 3: TODO 3: Timer configuration function

```c
void timer_init(void) {
    // IMPORTANT: Enable the clock FIRST -- all other
    // register writes are ignored until the clock is on.
    //
    // a) Enable TIM2 clock: RCC_APB1ENR |= (1 << 0)
    // b) Set prescaler: TIM2_PSC = 8399
    // c) Set auto-reload: TIM2_ARR = 1999
    // d) Enable update interrupt (peripheral level):
```

```
    //     TIM2_DIER |= TIM_DIER_UIE
    // e) Enable TIM2 in NVIC (CPU level):
    //     NVIC_ISER0 |= (1 << 28)
    // f) Start counting: TIM2_CR1 |= TIM_CR1_CEN
}
```

```
void timer_init(void) {
    /* Enable TIM2 clock (bit 0 of APB1ENR) - MUST be first! */
    RCC_APB1ENR |= (1 << 0);

    /* Prescaler: 84MHz / 8400 = 10kHz */
    TIM2_PSC = 8399;

    /* Auto-reload: 2000 ticks at 10kHz = 200ms */
    TIM2_ARR = 1999;

    /* Enable update interrupt (peripheral level) */
    TIM2_DIER |= TIM_DIER_UIE;

    /* Enable TIM2 in NVIC (CPU level, IRQ 28) */
    NVIC_ISER0 |= (1 << 28);

    /* Start counting */
    TIM2_CR1 |= TIM_CR1_CEN;
}
```

Common mistakes: Forgetting to enable the clock first (all other register writes are ignored), using 8400 instead of 8399 for PSC (off-by-one — timer counts from 0), forgetting NVIC enable (timer runs but ISR never fires).

4.4 Task 4: Timer ISR (`TIM2_IRQHandler`)

This is the real-time core. It executes every 200 ms, reads the sensor, checks thresholds, and fires the alarm if needed.

Why clear the flag FIRST?
When TIM2 overflows, it sets UIF in `TIM2_SR`. The NVIC sees this and calls the ISR. But the NVIC does **not** automatically clear UIF — only your code can do that.
When the ISR returns, the NVIC re-checks UIF. If it's still set (because you forgot to clear it, or cleared it too late), the NVIC immediately calls the ISR again, trapping the CPU in an infinite loop where the main loop never runs. Clearing UIF first prevents this.
`TIM2_SR &= ~TIM_SR_UIF;`

Listing 4: TODO 4: Timer interrupt service routine

```
void TIM2_IRQHandler(void) {
    // a) Clear UIF flag FIRST: TIM2_SR &= ~TIM_SR_UIF
    //
    // b) Call read_vitals()
    //     - If it returns 0 (sensor fault):
```

```
    //      set current_state = STATE_SENSOR_FAULT
    //      call alarm_on() as fail-safe
    //      set new_data_flag = 1, increment sample_count
    //      return early
    //
    // c) Call check_thresholds()
    // d) Increment sample_count
    // e) Set new_data_flag = 1
}
```

```
void TIM2_IRQHandler(void) {
    /* FIRST: Clear the update interrupt flag */
    TIM2_SR &= ~TIM_SR_UIF;

    /* Read sensor data via I2C */
    if (!read_vitals()) {
        current_state = STATE_SENSOR_FAULT;
        alarm_on();   /* Fail-safe */
        new_data_flag = 1;
        sample_count++;
        return;
    }

    /* Check thresholds (may activate/deactivate alarm) */
    check_thresholds();

    /* Update counters and signal main loop */
    sample_count++;
    new_data_flag = 1;
}
```

Timing analysis: The ISR executes: 1 register clear, 5 I2C transactions (each involving several register accesses for START/ADDR/DR/STOP), a few comparisons, and a GPIO write. Even with the I2C overhead, the entire ISR completes in well under 1 ms at 84 MHz — far within the 100 ms deadline. The alarm GPIO is set in the same ISR cycle as the detection.

5 Timing Verification

After your firmware compiles and runs, verify the timing requirements:

5.1 Verify 200 ms Sampling Interval (FR-01)

In the Renode monitor, you can check virtual time between samples. The UART output should show sample count incrementing at 5×/second. If you see 5 samples per second of virtual time in the UART analyzer, your timer is correctly configured.

5.2 Verify 100 ms Alarm Deadline (NFR-01)

Watch the UART output during the deterioration scenario. When the State changes from WARNING to CRITICAL (SpO2 drops below 90%), the alarm GPIO should activate **in the same ISR**

cycle as the detection. Since the ISR runs every 200 ms and the alarm activates within the ISR body (not deferred to the main loop), the worst-case alarm latency is:

$$t_{alarm} = t_{sample} + t_{ISR} \leq 200\,\text{ms} + 1\,\text{ms} \approx 200\,\text{ms from the actual physiological event}$$

But from the *detection* of the breach (the sensor read), the alarm fires within the same ISR execution, which is well within 100 ms.

Subtle point: The 200 ms sampling period means the system may not *detect* a breach for up to 200 ms after it physiologically occurs. This is the sampling latency, which is separate from the alarm processing latency. If the requirements demanded end-to-end latency < 100 ms, you would need to sample faster (e.g., every 50 ms). Discuss with your group: should FR-01 be revised?

Display refresh rate: The Week 2 specification requires different display refresh rates depending on system state (1 Hz for NORMAL, 2 Hz for WARNING/CRITICAL). This week's implementation uses a fixed 1 Hz rate for simplicity. State-dependent refresh rates will be implemented in Week 5 with Zephyr RTOS timers. The safety-critical alarm logic is **not affected** by this simplification — `alarm_on()` fires immediately in the ISR regardless of the display rate.

6 Deliberate Timing Failure Experiment

After your system works correctly, deliberately break the timing to observe the consequences.

6.1 Experiment A: Printf in the ISR

Add a `uart_puts("ISR fired\r\n")` call inside your `TIM2_IRQHandler`. At 115200 baud, printing 12 characters takes ∼1 ms. This is still fast, but what happens if you print a longer diagnostic message (say 200 characters, ∼17 ms)?

[Observe and note what happens to the sampling regularity and system responsiveness]

Printing 200 characters at 115200 baud takes ∼17 ms (200×10 bits $\div$ 115200 $\approx$ 17.4 ms). This blocks the ISR for 17 ms per invocation. Since the ISR fires every 200 ms, the CPU spends ∼8.7% of its time in the ISR just doing UART I/O. The alarm still fires within the deadline (17 ms $<$ 100 ms), but the system is now fragile: add more work and you approach the limit. More critically, if another interrupt needs to fire during those 17 ms, it will be delayed. This is why **UART output should never be in a safety-critical ISR** — it should be in the main loop (as our architecture correctly does).

6.2 Experiment B: Reverting to Delay Loop

Temporarily disable the timer interrupt and move the sensor reading back into the main loop with a `delay_ms(200)` call (like Week 3 but with real sensors). Start the deterioration scenario. Does

the alarm still fire within 100 ms of detection?

[Your observations]

With the delay loop, the sensor read and threshold check happen sequentially in the main loop. The alarm will still fire quickly *after detection*, but the detection itself is unreliable: the `delay_ms()` loop's actual duration depends on compiler optimization, and any UART printing in the loop adds to the cycle time. The key problem is that UART printing (50+ characters at 1 Hz) takes ∼4 ms, making the actual loop time ∼204 ms instead of 200 ms. Over 100 iterations, this drift accumulates to ∼400 ms of error. With the timer interrupt, sampling is hardware-guaranteed at exactly 200 ms regardless of what the main loop is doing.

7 Reflection Questions

Q1: Why are the global variables (`latest_hr`, `latest_spo2`, etc.) declared as `volatile`? What could go wrong if they weren't?

[Your answer]

Without `volatile`, the compiler may optimize reads of these variables in the main loop: it might cache the value in a register and never re-read from memory, assuming it hasn't changed. But the ISR modifies them asynchronously. So the main loop could see stale data indefinitely — e.g., always displaying SpO2=97% even after the ISR has updated it to 85%. With `volatile`, the compiler is forced to read from memory every time the variable is accessed, ensuring the main loop sees the ISR's updates. This is a classic shared-memory concurrency issue in embedded systems.

Q2: In our architecture, the alarm is activated *inside the ISR*. Could we instead set a flag in the ISR and activate the alarm in the main loop? What are the timing implications?

[Your answer]

If we deferred alarm activation to the main loop via a flag, the latency would depend on how quickly the main loop checks the flag. If the main loop is busy with a long UART print (∼4 ms) or any other processing, the alarm could be delayed by that amount. Worse, if the main loop

has a bug that causes it to hang, the alarm would never fire. By activating the alarm directly in the ISR, we guarantee the worst-case latency is bounded by the ISR execution time (< 1 ms), regardless of what the main loop is doing. This is a fundamental principle of safety-critical design: **never defer safety actions to lower-priority code**.

Q3: The current system has only one interrupt (TIM2). Next week, you'll have multiple RTOS tasks with different priorities. What happens if a lower-priority task is running when the timer fires? What if a higher-priority interrupt fires during TIM2's ISR?

[Your answer]

On ARM Cortex-M4 with NVIC: if a lower-priority task (or main loop code) is running when TIM2 fires, it is **preempted immediately** — the hardware saves context and jumps to the ISR. The lower-priority code resumes after the ISR returns. This is why our alarm works reliably even if the main loop is busy. Conversely, if a higher-priority interrupt fires during TIM2's ISR, TIM2's ISR is preempted (nested interrupts). This could delay our alarm processing. In Week 5 with Zephyr RTOS, we'll manage this with task priorities rather than raw interrupt priorities, which gives more structured control over preemption.

8 Looking Ahead — Week 5 Preview

Your system now works, but it has a rigid architecture: everything real-time is crammed into a single ISR. This doesn't scale. What if you needed to:

- Add a second sensor with different sampling rate?
- Implement state-dependent display refresh rates (1 Hz vs. 2 Hz)?
- Add data filtering that takes significant computation?

Next week, you will port SafePulse to **Zephyr RTOS**, decomposing the monolithic ISR into cooperating tasks with different priorities. You'll see how priority inversion can break your alarm deadline — and how to prevent it.

Keep your working code! The Week 5 lab will start from this week's solution and restructure it into RTOS tasks. Make sure everything compiles and the alarm triggers correctly before leaving.